Sleep/Wake Scoring — Installation & Guide

A semi-automated GUI for classifying sleep/wake states from EEG + EMG (and optional video/motion) recordings. A Random Forest model proposes a state for every 4-second epoch; a human reviewer inspects the spectrogram, EMG, theta/delta ratio, and (optionally) video, then corrects the predictions. Corrected labels can be fed back to retrain the model.

This document covers: how to install the package, how to prepare and lay out your data, the configuration file, the full scoring workflow, the GUI controls, and a reference for every file type and data structure the engine consumes or produces.

Package source: `neuroscience_sleep_scoring`

---

1. What the pipeline does

```
 RAW DATA                  PREPROCESSING                  SCORING
 (per experiment)          (run ONCE per experiment)      (run per acquisition)

 AD0_*.mat  (EEG ch0)
 AD2_*.mat  (EEG ch2)  →   extract_data.py            →   New_SWS.py
 AD3_*.mat  (EMG)          • pick acquisitions            • load downsampled
EEG/EMG
 trigger_times.mat         • bandpass + downsample        • build feature
vectors
 *timestamp*.csv  (video)  • compute normVal              • Random Forest
prediction
 *motion*.csv  (video)     • (optional) combine video     • review/correct
 *.mp4 / *.avi
                           • (optional) make EDF          • save StatesAcq*.npy

                           • (optional) velocity array    • (optional)retrain
```

Two scripts you run directly:

Script	When	Frequency
`extract_data.py`	After acquiring data, before scoring	**Once per experiment**
`New_SWS.py`	To score / correct an acquisition	Every scoring session

---

2. Installation

2.1 Prerequisites

- **Python 3.6+** (3.8–3.10 recommended; see version pitfalls in §2.4).
- **A real GUI display.** The interface uses an interactive Matplotlib backend (`plt.ion()`, `waitforbuttonpress`) plus OpenCV windows for video. It will **not** run on a headless server. Use a local Mac/Windows/Linux desktop, or X-forwarding / a virtual desktop (VNC). On macOS the `MacOSX` or `TkAgg` backend works; on Linux install `python3-tk`.
- **Access to your raw data.** Paths are configured per-experiment (see §4), but a few legacy helper functions still contain hardcoded `/Volumes/...` paths (see §2.4).

2.2 Python dependencies

Declared in `setup.py`:

```
numpy  scipy  matplotlib  psutil  pandas  natsort  opencv-python  pyedflib
scikit-learn  seaborn
```

2.3 Install steps

```
# 1. Clone

git clone https://github.com/YaoChenLabWashU/neuroscience_sleep_scoring.git
cd neuroscience_sleep_scoring

# 2. (Recommended) create an environment

conda create -n sleepscore python=3.9
conda activate sleepscore

# 3. Install the package + dependencies

pip install -e .

# 4. Make the PKA_Sleep dependency importable (see warning below)
```

2.4 ⚠️ Paths

> **Hardcoded NAS paths in a few functions.** `New_SWS.build_model`, `SWS_utils.get_eeg_file`, and parts of `extract_data` contain literal `/Volumes/yaochen/Active/...` paths. The main `extract_data → New_SWS` flow is driven entirely by the Score_Settings file and does **not** depend on these, but the legacy/bootstrap helpers do. Edit them as needed.

---

3. Required inputs: file types & data structures

3.1 Raw electrophysiology: `AD<channel>_<acq>.mat`

One MATLAB `.mat` file per channel per acquisition, named by channel and acquisition number:

File	Meaning
`AD0_<n>.mat`	EEG channel 0 (used for the spectrogram and the model)
`AD2_<n>.mat`	EEG channel 2 (second EEG; enables the 2-channel model)
`AD3_<n>.mat`	EMG

`<n>` is the **acquisition number** (an integer, e.g. `AD0_4.mat`).

Internal structure. Each file contains a single MATLAB struct variable whose name equals the filename stem (e.g. file `AD0_4.mat` → variable `AD0_4`). The raw 1-D sample vector is reached through nested indexing, and acquisition metadata lives in a `UserData.headerString` field:

```
mat   = scipy.io.loadmat('AD0_4.mat')
data  = mat['AD0_4'][0][0][0][0]          # 1-D numpy array of samples
hdr   = mat['AD0_4']['UserData'].item()['headerString'].item()
fs    = int(hdr[hdr.find('inputRate=')+10:].split('\r')[0]) # sampling rate
```

- Sampling rate (`fs`, e.g. 400 Hz) is declared in your config and must match the rig. The pipeline downsamples to `fsd` (e.g. 200 Hz).
- All `AD*` files for one experiment live in a single directory (`rawdat_dir`).

3.2 Acquisition start times: `trigger_times.mat` (optional but recommended)

If present in `rawdat_dir`, this file provides exact start datetimes for each acquisition (variable `trigger_times`, indexed by acquisition order). It is used to align video/motion to ephys.

- **If absent**, the code falls back to estimating each acquisition's start from the `.mat` file's filesystem modification time minus its duration. This is less reliable; prefer providing `trigger_times.mat` whenever video is used.

3.3 Video: `<basename><n>.mp4` (or `.avi`)

Optional. Required only if `vid = 1`. One video per recording segment, named `<basename><n>.<ext>` (e.g. `ltFLiPAKARmemEEGEMG0095_3.mp4`). Stored in `video_dir`. `.mp4` is searched first, then `.avi`.

3.4 Bonsai timestamp CSVs: `*timestamp<n>.csv`

Optional (needed for video and/or motion). One CSV per video segment, stored in `csv_dir`. Format: a single column of ISO-8601 frame timestamps, no header:

```
2023-04-12T15:04:01.2345678+00:00
2023-04-12T15:04:01.2680000+00:00
...
```

The `<n>` suffix in the filename ties each timestamp file to its video and motion file. Timestamps are auto-adjusted against the file's modification time to correct clock offset.

3.5 Motion CSVs: `*motion<n>.csv`

Optional (needed only if `movement = 1`). One per segment, in `csv_dir`. Two supported formats, selected by the `DLC` config flag:

- **DeepLabCut (`DLC = 1`)** — standard multi-row DLC output. The body part named by `DLC Label` (e.g. `center`) is read using its `_x`, `_y`, and `_likelihood` columns. Frames with likelihood < 0.8 are flagged as low-quality.
- **Bonsai raw (`DLC = 0`)** — a headerless CSV with either `X,Y` columns or `Timestamps,X,Y` columns.

Velocity per epoch is computed as the Euclidean displacement of the tracked point between epoch boundaries.

3.6 Directory layout (recommended)

```
<rawdat_dir>/                          # all raw ephys for one experiment
    AD0_4.mat  AD0_5.mat  ...
    AD2_4.mat  AD2_5.mat  ...
    AD3_4.mat  AD3_5.mat  ...
    trigger_times.mat                  # optional

<video_dir>/                           # if vid = 1
    <basename>3.mp4  <basename>4.mp4   ...

<csv_dir>/                             # if vid/movement = 1
    *timestamp3.csv  *timestamp4.csv   ...
    *motion3.csv     *motion4.csv      ...

<savedir>/                             # created by extract_data.py
    (downsampled data + scoring outputs — see §6)

<model_dir>/                           # the shared model + logs
    EEG_2chan_EMG_nomovement.joblib
    mouse_2chan_model.pkl
    *_scoringlog.txt
```

---

4. Configuration: `Score_Settings.json`

Everything is driven by a single JSON file passed as the only command-line argument. Keep a personal copy per experiment. Every key:

Key	Type	Meaning
`basename`	string	Experiment name; used to build video/normVal filenames.
`rawdat_dir`	path	Folder containing the raw `AD*.mat` (and `trigger_times.mat`).

Key	Type	Meaning
`model_dir`	path	Folder holding the shared `.joblib` model and `*_model.pkl`.
`video_dir`	path	Folder with video files (if `vid`).
`csv_dir`	path	Folder with Bonsai `timestamp`/`motion` CSVs.
`modellog_dir`	path	Where the model scoring log is written (usually = `model_dir`).
`personallog_dir`	path	Where your personal scoring log (`personal_scoringlog.csv`) is written.
`species`	string	`"mouse"`
`mod_name`	string	Model name stem (e.g. `"mouse_2chan"`). Determines `*_model.pkl` filename.
`mouse_name`	string	Animal ID (not the experiment name).
`epochlen`	int	Epoch/bin length in seconds. **Minimum 4**; the model and features assume 4.
`fs`	int	Raw sampling rate of the `.mat` data (e.g. 400).
`fsd`	int	Target downsampled rate (e.g. 200).
`emg`	0/1	1 if EMG (`AD3`) was recorded.
`vid`	0/1	1 to enable video review.
`movement`	0/1	1 to use motion/velocity as a feature & plot.
`EEG channel`	list[int]	EEG channels to use, e.g. `[0, 2]`. **Must be a list.** Length 2 → 2-channel model.
`EMG channel`	int	The EMG channel number (e.g. 3).
`Acquisition`	list[int]	Acquisitions to process. Auto-filled by `extract_data.py`.
`Filter High`	int	Band-pass high cutoff (Hz) for EEG/EMG (e.g. 100).
`Filter Low`	float	Band-pass low cutoff (Hz) (e.g. 0.5).
`savedir`	path	Where downsampled data and scoring outputs go.
`Bonsai Version`	int	Which Bonsai workflow produced the video/CSV files.
`DLC`	0/1	1 if motion CSVs are DeepLabCut output; 0 for raw Bonsai.
`DLC Label`	string	Body part to track for velocity (e.g. `"center"`).
`Minimum_Frequency`	int	Lower bound (Hz) of the displayed spectrogram (e.g. 1).
`Maximum_Frequency`	int	Upper bound (Hz) of the displayed spectrogram (e.g. 20).
`vmin`	number or `"None"`	Spectrogram color floor (dB). `"None"` = auto.
`vmax`	number or `"None"`	Spectrogram color ceiling (dB). `"None"` = auto.

5. Usage workflow

Step 1 — Preprocess

```
cd neuroscience_sleep_scoring
python extract_data.py /path/to/Score_Settings.json
```

This runs, in order:

1. **`choosing_acquisition`** — lists the `AD0_*.mat` files and asks whether to add **all** acquisitions, choose them **individually**, or include only those above a **minimum length**. Your selection is written back into the config's `Acquisition` list.
2. **`downsample_filter`** — band-pass filters (Butterworth, order 3, using `Filter Low`–`Filter High`) and resamples each acquisition from `fs` to `fsd`. EMG is additionally 10 Hz high-passed. Saves the downsampled arrays (see §6.1).
3. **`get_normalizing_value`** — computes the median total spectral power across acquisitions per EEG channel and saves it as `<basename>_normVal.npy`. This value normalizes band-power features so they are comparable across sessions.
4. **EDF export** (prompted, optional) — bundles EEG+EMG into `.edf` files for external viewers.
5. **Video/motion** (only if `movement = 1`) — `combine_bonsai_data` merges all timestamp and motion CSVs into `All_timestamps.pkl` / `All_movement.pkl`, and optionally builds the full `velocity_vector.npy`.

Step 2 — Score / correct (per acquisition)

```
python New_SWS.py /path/to/Score_Settings.json
```

The engine then:

1. Lists available acquisitions and which already have a saved `State` file.
2. Prompts: **which acquisition** to score.
3. Splits the acquisition into ≤1-hour segments (most are a single segment).
4. For each segment, prompts: **check existing scoring (`c`)** or **score a new dataset (`s`)**:
    - **`s` (score new):** optionally runs the Random Forest to predict states, saves `model_prediction_Acq<a>_hr<h>.npy`, then opens the GUI pre-filled with the prediction for you to correct. (Choosing not to use the model opens a blank, all-zero scoring you fill in manually.)
    - **`c` (check/fix):** loads the existing `StatesAcq<a>_hr<h>.npy` and opens the GUI so you can adjust it. If any epoch is still unscored (state 0) it warns you and offers to go back.
5. On exit, prompts whether to **update the model** (retrain with this segment's corrected labels) and whether to **update your personal log**.

---

6. The GUI: layout & controls

When scoring opens, you get two figure windows:

Figure 1 (the main figure) — 5 stacked panels:

1. Spectrogram of EEG channel 0 (frequency band set by `Minimum/Maximum_Frequency`).
2. **Predicted states** strip (the row you click to correct) with a black model-confidence trace overlaid.
3. Spectrogram of EEG channel 2.
4. Velocity (or "No Movement Available").
5. EMG amplitude.

Figure 2 (the zoomed/raw figure) — synchronized detail panels for the EMG, theta/delta ratio, and (if available) velocity around the current epoch, plus a moving marker line.

Controls

Action	How
Re-center / inspect an epoch	Click once inside the **spectrogram** (panel 1). The raw-trace figure (and video, if enabled) jumps to that epoch ± surrounding epochs.
Correct a run of epochs	Click the **start** epoch in the predicted-states strip (panel 2), then click the **end** epoch. Switch to the terminal and type the new state number (1–4) when prompted. The strip recolors and `StatesAcq*.npy` is re-saved immediately.
Toggle a crosshair line	With Figure 2 focused, press `l` to drop/erase a vertical alignment line across panels.
Score a single epoch (manual mode)	Keys `1`,`2`,`3`,`4` set the current epoch's state.
Exit a video clip early	Press `v`.
Finish this segment	Press `d` (done). You'll then be asked about saving / updating the model.
Quit immediately	Press `q` (closes everything and exits).

All keys are lowercase (no Shift). If a click "does nothing," make sure Figure 2 (not the terminal or video window) has focus. Don't toggle the line while hovering the motion panel.

---

7. Sleep-state encoding

States are integers. In the GUI they are entered as keystrokes `1`–`4` and color-coded in the predicted-states strip:

State	Color	Conventional meaning*
`0`	grey / white	**Unscored** (placeholder; must be resolved before retraining)
`1`	green	REM / active sleep
`2`	blue	NREM / slow-wave sleep
`3`	red	Wake
`4`	purple	Fourth state (e.g. active vs. quiet wake / transitional)

---

8. Data-structure reference

8.1 Generated by `extract_data.py` (written to `savedir`)

File	Shape / type	Description
`downsampEEG_Acq<a>.npy`	1-D float array	Full downsampled EEG for acquisition `a` (used to get total length).
`downsampEMG_Acq<a>.npy`	1-D float array	Full downsampled EMG for acquisition `a`.
`AD<chan>_downsampled/downsampEEG_Acq<a>_hr<h>.npy`	1-D float array	Per-hour EEG segment for channel `chan`, trimmed to a whole number of epochs.
`downsampEMG_Acq<a>_hr<h>.npy`	1-D float array	Per-hour EMG segment.
`AD<chan>_downsampled/<basename>_normVal.npy`	scalar	Median total power; normalizes band-power features.
`All_timestamps.pkl`	pandas DataFrame	Columns `Timestamps`, `Filename` — all video frame times.

File	Shape / type	Description
`All_movement.pkl`	pandas DataFrame	Columns `Timestamps, X, Y, Likelihood, Filename` — all tracked positions.
`velocity_vector.npy`	2×N array	Row 0 = per-bin velocity, row 1 = bin time (s).
`*.edf`	EDF	Optional EEG+EMG export for external viewers.

8.2 Generated by scoring (`New_SWS.py`, written to `savedir`)

File	Shape / type	Description
`StatesAcq<a>_hr<h>.npy`	1-D int array, one entry per epoch	**The primary output** — your final/working sleep-state labels. Re-saved every time you correct a run of bins.
`model_prediction_Acq<a>_hr<h>.npy`	1-D int array	The raw Random Forest prediction (before your corrections), saved when you score in model mode.

8.3 Model

File	Type	Description
`<mod_name>_model.pkl`	pickled pandas DataFrame	Accumulated **training table**: every feature column plus `State`, `animal_name`. Grows each time you "update the model".

8.4 Logs

File	Type	Description
`<mod_name>_scoringlog.txt`	text	Who scored what, when, and in which mode (`corrected` / `scored with ML model` / `scored in legacy mode`).
`personal_scoringlog.csv`	CSV	Columns `Date, Mouse Name, Acquisition, State Array Location`.

8.5 In-memory structures

`d` (settings dict) — the parsed `Score_Settings.json`, threaded through nearly every function.

`FeatureDict` — built by `SWS_utils.build_feature_dict`. One key per feature, each a 1-D array of length *num_epochs*. Keys are generated per EEG channel `c` (e.g. `EEGChannel0`, `EEGChannel2`):

- `EMGvar` — EMG variance per epoch (if EMG present).
- `<c>var` — EEG variance per epoch.
- `<c>Delta`, `<c>Theta`, `<c>Alpha`, `<c>BroadTheta`, `<c>Fire` — band powers (0.5–4, 5–8, 8–12, 2–16, 4–20 Hz), normalized by `normVal`.
- `<c>nb` — narrow-band theta = Theta / BroadTheta.
- `<c>thet_delt` — Theta / Delta ratio.
- `<c>delta_pre/post`, `…_pre2/post2`, `…_pre3/post3` — the delta feature shifted ±1/2/3 epochs (gives the model temporal context). Same for `theta_*` and `nb_pre/post`.
- `Velocity` — per-epoch velocity (if `movement`).
- `State` — added after scoring; the supervised target.

---

9. Module / function map

What each file is responsible for, and the key functions the engine calls.

`New_SWS.py` — scoring engine (entry point)

- `load_data_for_sw(config)` → `start_swscoring(d)` — top-level driver.
- `start_swscoring(d)` — acquisition/segment loop; prompts; orchestrates predict → review → save → optional retrain.
- `display_and_fix_scoring(...)` — builds the figures, wires up the cursor, runs the interactive event loop, writes `StatesAcq*.npy`.
- `update_model(d, FeatureDict)` — appends corrected labels to the training table and retrains the `.joblib`.
- `model_log(...)`, `personal_log(...)` — write the audit logs.

`extract_data.py` — preprocessing (run once)

- `choosing_acquisition`, `pulling_acqs` — discover/select acquisitions.
- `downsample_filter` — filter + resample → downsampled `.npy`.
- `get_normalizing_value` — compute `normVal`.
- `combine_bonsai_data`, `make_full_velocity_array` — video/motion consolidation.
- `make_edf_file`, `save_to_edf`, `get_phys_fs` — optional EDF export & header parsing.

`SWS_utils.py` — shared library (~50 functions)

- **Features:** `build_feature_dict`, `generate_signal`, `bandPower`, `plot_spectrogram`/`my_specgram`, `post_pre`, `get_ThD`, `get_total_power`, `prepare_feature_data`, `get_xfeatures`, `adjust_movement`.
- **Model:** `update_sleep_df`, `update_sleep_model`, `retrain_model`, `random_forest_classifier`, `build_joblib_name`, `load_joblib`, `fix_states`, `define_microarousals`.
- **Figures/interaction:** `create_prediction_figure`, `plot_predicted`, `create_zoomed_fig`, `plot_zoomed_data`, `update_raw_trace`, `make_marker`, `add_buffer`, `clear_bins`, `correct_bins`, `print_instructions`.
- **Video/timestamps/motion:** `load_video`, `pull_up_movie`, `get_videofn_from_csv`, `timestamp_extracting`, `pulling_timestamp`, `initialize_vid_and_move`, `movement_extracting`, `movement_processing`, `get_movement_segs`, `sort_files`, `sort_timestamp_files`, `get_AcqStart`.
- **DLC:** `movement_extracting` (DLC branch), `DLC_check_fig`, `rename_DLC_csvs`, `transfer_DLC_files`.

`SW_Cursor.py` — interaction classes

- `Cursor` — full interaction: re-center clicks on the spectrogram, two-click bin selection on the states strip, crosshair, and the `d`/`l` key handlers. Sets flags (`replot`, `change_bins`, `DONE`) that the `New_SWS` event loop reads.
- `ScoringCursor` — a trimmed variant.

---

10. Troubleshooting

Symptom	Likely cause / fix
`ModuleNotFoundError: PKA_Sleep` on import	The undeclared dependency isn't on the path. Install/add `PKA_Sleep`
`AttributeError: module 'numpy' has no attribute 'float'` after an odd keypress	NumPy ≥1.24. Pin `numpy<1.24` or patch `np.float('nan')`→`float('nan')`
`ImportError: cannot import name 'simpson'`	SciPy too old. Upgrade SciPy or alias `simps`
Figure opens but clicks do nothing	Wrong window focused — click into **Figure 2**, not the terminal/video.

Symptom	Likely cause / fix
"No videos found" / timestamp errors	Check `video_dir`/`csv_dir`, filename suffixes (`<basename><n>`, `*timestamp<n>`, `*motion<n>`), and that counts of timestamp vs motion files match.
Wrong/old labels reload when checking	You're loading `StatesAcq<a>_hr<h>.npy` — confirm the acquisition/hour and that prior corrections were saved (they save automatically on each bin edit).
Model not found in score mode	The `.joblib` whose name matches your `EEG channel`/`emg`/`movement` config isn't in `model_dir`.
